

3. The Token Economics of Coding Agents

From autocomplete complements to metered autonomy

THE INTELLIGENCE ECONOMY · A WISDOM HILL RESEARCH SERIES
Wisdom Hill Research 2026-08-24

Table of contents

Executive Summary 2

Introduction 3

Part I. The Coding Assistant Era 3

 The Complementarity Regime 3

Part II. The Tokenmaxxing Era 5

 The Unmetered Substitution Experiment 5

Part III. The Maturation Era 10

 The Metered Equilibrium 10

Part IV. Monitoring and Implications 15

Appendices 16

 Appendix A — The 80% Concentration Threshold: Derivation 16

 Appendix B — Data Conventions and Known Disputes 17

 Appendix C — Glossary 17

References 17

Executive Summary

The defining empirical puzzle of the AI industry in 2025–26 is the explosion of token consumption in software development. Anthropic’s annualized run-rate revenue rose from roughly \$9 billion at the end of 2025 to a company-confirmed \$47 billion by mid-May 2026 (disclosed in its \$65 billion Series H announcement) — a 5.2x increase over roughly nineteen weeks, implying a doubling period of about eight weeks — with agentic coding products prominently featured among the growth drivers Anthropic itself cites (Claude Code, Cowork), though the company has not attributed a specific share of growth to them. This report argues that the right lens for understanding both the explosion and its likely future path is not model capability per se but **token economics**: the cost, allocation, and productivity of tokens per verified unit of software output.

We organize the history and forecast into three stages. **Stage One (2022–2024), the Coding Assistant era**, is the complementarity regime documented in the NBER literature: AI tools multiplied raw code production (commits +180% for autonomous agents) but gains attenuated sharply down the production hierarchy (releases +30%), with an estimated elasticity of substitution between AI and human effort of just 0.25 — humans, specifically human verification, were the binding constraint. **Stage Two (2025–H1 2026), the Tokenmaxxing era**, was an unmetered substitution experiment led by a small vanguard of frontier developers who abandoned IDEs, stopped reading code, and ran fleets of agents in loops — burning tokens at rates approaching, and in extreme cases exceeding, fully loaded labor costs. Their trial-and-error built the verification-automation stack (autonomous loops, adversarial review, model-native self-verification) that is now eroding the very complementarity the NBER parameter described. **Stage Three (H2 2026 onward), the Maturation era**, is the metered equilibrium: CFO discipline arrives, spending becomes governed and measured, the unproductive tail of the usage distribution is compressed, and growth shifts to the broad middle of the developer population adopting the vanguard’s now-codified practices.

Our central quantitative findings: (1) the extensive margin of adoption is largely saturated — roughly 90% of professional developers worldwide report regular use of AI coding tools (JetBrains, 2026) and fewer than one in ten uses none — while daily-intensity use sits near 51% (Stack Overflow, 2025), so all forecast-relevant information now lives in the *intensity distribution* of usage rather than in headcount adoption; (2) whether enterprise rationalization reduces or grows total tokens depends on a single parameter — the share of tokens consumed by heavy users — with an analytical breakeven at 80%; (3) CFO constraints bind on dollars, not tokens, so volume growth will substantially outrun dollar growth as prices deflate and routing matures; (4) the largest remaining cost lever is not per-token price but loop count (N), where generational model improvements deliver step-function collapses — GitLab reports Fable 5 completing in a single pass systems that previously required days of iteration, and GitHub reports equivalent work finished with fewer tool calls and lower token consumption than prior Opus-tier models; and (5) the delegable task frontier is expanding along two independent axes — model capability and specification quality — with agentized spec elicitation accelerating the latter.

Bottom line: total token consumption does not saturate within the forecast horizon. Dollar spending growth decelerates sharply in H2 2026 as the rationalization cycle plays out, but

this is a normalization of revenue quality, not demand destruction. Margin migrates from the model layer toward the routing, measurement, and governance layers — the control plane of the token economy. The principal reversal risk is vendor success in per-task pricing, which would let the model layer recapture the loop-efficiency surplus (\$4.2).

Introduction

Three facts frame this report. First, the revenue facts: Anthropic’s run-rate revenue path — \$87 million (January 2024), \$1 billion (December 2024), \$9 billion (end of 2025), \$14 billion (February 2026), \$19 billion (March), \$30 billion (April), and a company-confirmed \$47 billion by mid-May 2026 (Series H disclosure) — represents organic scaling without precedent in enterprise software. Claude Code alone went from public launch (May 2025) to more than \$2.5 billion in annualized revenue within nine months. Second, the consumption facts: workloads shifted from single completions to multi-step autonomous sessions, and the resulting volumes now register in corporate disclosures — Meta’s internal memo recorded 73.7 trillion tokens consumed by roughly 6,000 employees in about thirty days (\$2.3). Per-token prices for fixed capability fell more than 98% over a period in which enterprise AI bills nonetheless rose to labor-cost scale. Third, the credibility caveat: run-rate revenue is a single-month snapshot multiplied by twelve, and critics (notably Ed Zitron) have documented tensions between Anthropic’s quarterly revenue disclosures and its sequence of ARR announcements. Directionally the growth is real; the precise figures deserve a discount.

The thesis of this report is that these facts are best understood through a production-function lens in which the relevant unit is not the token but the **verified unit of software output**, and the relevant price is the **cost per completed, trusted task**. Each of the three stages below is defined by a different answer to the question: *what is the binding constraint on converting tokens into verified output?* In Stage One, it was human verification. In Stage Two, it was compute supply and the absence of metering. In Stage Three, it will be measured return on investment.

Part I. The Coding Assistant Era

The Complementarity Regime

1.1 What the NBER Evidence Actually Measured

The most rigorous economic evidence on AI-assisted software development is Demirer, Musolff and Yang (NBER Working Paper 35275, 2026). Using matched event-study designs over more than 100,000 GitHub developers linked to AI-usage telemetry, the paper estimates cumulative commit increases of +40% for autocomplete tools, +140% for interactive coding agents, and +180% for autonomous coding agents. The striking result is not the gains but their **attenuation up the production hierarchy**: the 180% commit increase shrinks to roughly +50% at the project level and +30% at the release level, and across four major app marketplaces, new app counts rose only modestly while total app usage did not rise at all.

The paper interprets this pattern through a weak-link production function and estimates an elasticity of substitution between AI and human effort of $\sigma \approx 0.25$ — strong complementarity, close to a Leontief technology. The economic meaning is stark: when σ is this low, additional AI input barely substitutes for human input; output is determined by the scarcest complement in the chain. In Stage One, that scarcest complement was human judgment applied to verification, integration, and release decisions. The popular intuition — “AI does 90% of the coding, but the remaining 10% absorbs 90% of the time” — is the folk version of this parameter.

1.2 The Dating Problem: A Lucas Critique

A critical limitation, however, concerns identification timing. The experimental variation underpinning the structural elasticity estimate derives from GitHub Copilot **autocomplete** deployments dating to the second half of 2022 (June 21 – December 26, 2022). Autocomplete is the *least* substitutable generation of AI coding tools: it accelerates typing — one step in the production process — while leaving design, verification, and integration untouched. An elasticity estimated on that technology measures the substitutability of a typing accelerator, not of an autonomous agent.

This is a textbook application of the Lucas critique: structural parameters estimated under one technological regime do not transfer to another. Two implications follow. First, $\sigma = 0.25$ should be read as a **lower bound** for the agentic era; the paper’s own commit gradient (+40% $\rightarrow$ +140% $\rightarrow$ +180% across tool generations) is indirect evidence that substitutability rises with autonomy. Second — and this preserves the paper’s enduring value — the *attenuation pattern* (commits $\rightarrow$ releases) was measured on recent data including autonomous agents, so the existence of a weak link is a current fact even if its tightness (0.25) is a dated number. The history of Stages Two and Three, as told below, is essentially the history of attacking that weak link.

1.3 The Verification Bottleneck in Evidence

Production telemetry confirms where the weak link sat. Faros AI, tracking 1,255 teams and over 10,000 developers, found that AI-adopting teams completed 21% more tasks and merged 98% more pull requests — while PR review time rose 91%. The bottleneck did not disappear; it relocated from writing code to verifying it. Code-quality data points the same direction: GitClear’s analysis of 211 million changed lines found code churn (lines re-edited within two weeks) nearly doubling from 3.1% to 5.7%, copy-pasted code rising 48% between 2020 and 2024, and refactored code collapsing from 24.1% to 9.5% of changes — the first period on record in which copy-paste exceeded refactoring. Survey evidence completes the picture: adoption is near-universal (roughly 90% of professional developers worldwide report regular use; JetBrains, 2026), yet only 48% consistently verify output before committing and 88% report negative downstream impacts; and the debugging burden is rising on every instrument that measures it — 38% of developers in Sonar’s 2026 survey report that AI-generated code demands more review effort, 45% in Stack Overflow’s 2025 survey report spending more time debugging AI-generated code, and 67% in Harness’s State of Software Delivery report the same increased debugging burden.

The era also produced a foundational methodological warning. In METR’s randomized controlled trial, experienced developers *estimated* that AI assistance made them 20% faster while *measured* completion times were 19% slower — a 39-point perception gap. (A 2026 follow-up on a subset of the original participants estimated a reversal to roughly 18% faster, but with a

confidence interval spanning -38% to $+9\%$ and acknowledged selection effects; METR itself characterizes the speedup evidence as weak.) Any claim in this industry that rests on developers' *felt* productivity must be discounted accordingly.

1.4 The Macro Non-Result

At the aggregate level, Stage One left an ambiguous footprint. By early 2026, US labor productivity was running about 2.2% above the CBO's pre-pandemic trajectory, growing near 2.8% annually — enough that previously skeptical economists began attributing part of the acceleration to AI: Jason Furman came to concur with Erik Brynjolfsson's reading of the BLS benchmark revisions, which cut employment by roughly 403,000 while real GDP held near 3.7%. But total factor productivity — the pure efficiency residual — *decelerated* from 1.5% to 0.8% in 2025. The acceleration, in other words, was arriving through the capital-deepening channel (massive AI investment), not the efficiency channel. Productivity gains that are real but diffuse do not show up as TFP, and — as Stage Three will make consequential — they do not survive a CFO's quarterly review either.

Part II. The Tokenmaxxing Era

The Unmetered Substitution Experiment

2.1 The Frontier Vanguard and the Death of the IDE

Stage Two was led not by the median developer but by a small vanguard that deliberately broke the complementarity regime. Its most articulate spokesman, Steve Yegge, mapped the transition as an eight-stage “Evolution of the Programmer”: from autocomplete (Stage 1), through agents-in-IDE with permissions (Stages 2–4), to CLI-based single agents where “diffs scroll by” largely unread (Stage 5), to multi-agent parallelism (Stage 6), to hand-managing fleets of 10+ agents (Stage 7), to building one's own orchestrator (Stage 8). Yegge's Gas Town, open-sourced on January 1, 2026, orchestrates 20–30 parallel Claude Code instances under a role hierarchy, burns roughly \$100 per hour at full capacity, and exhausted three Claude Max accounts in its launch week. His deliberately provocative formulation — that using an IDE in 2026 marks a bad engineer — became the era's slogan.

The behavioral shift the vanguard pioneered was the move from *code review* to *outcome review*: stop reading the generated code, inspect the behavior of the artifact, and feed back improvement requests. Peter Steinberger's admission — “I don't read much code anymore” — circulated widely; Addy Osmani documented that by early 2026 more than 30% of senior developers reported shipping mostly AI-generated code, with the binding constraint shifting to inference latency rather than typing. In production-function terms, outcome review changes the verification object from process (code, where human time scales with AI output) to product (behavior, where human time is roughly fixed) — precisely the change that raises the effective elasticity of substitution and unwinds the $\sigma = 0.25$ regime.

Two qualifications are essential. First, the vanguard was tiny relative to the population: Anthropic's own 2026 Agentic Coding Trends Report found developers using AI in roughly 60% of their work while being able to *fully delegate* only 0–20% of tasks — the “delegation

gap.” Second, the vanguard was supply-constrained, not demand-constrained: rate limits bound constantly, Anthropic publicly acknowledged infrastructure strain affecting peak-hour reliability, and the heaviest users stacked multiple subscription accounts. Today’s “maximum delegation” is a rationed ceiling, not a demand ceiling.

2.2 The Verification Automation Stack

The vanguard’s lasting contribution was converting its trial-and-error into a three-layer verification-automation stack.

Layer 1: Loop infrastructure. Geoffrey Huntley’s “Ralph Wiggum” technique (July 2025) — in its purest form a bash loop feeding a prompt file to Claude Code until the specification is satisfied — demonstrated that autonomous agents could produce 1,000+ commits overnight at roughly \$10/hour of metered API spend. Its key insight was fresh context per iteration: the agent confronts its own output and mistakes each cycle. Structured variants decomposed work into plan → implement → test → verify → PR phases, each an independent agent loop with checkpoints. The pattern was then absorbed as a first-class product primitive: Anthropic shipped an official ralph-wiggum plugin for Claude Code (December 2025), and OpenAI shipped the /goal command in Codex CLI 0.128.0 (April 30, 2026) — an autonomous plan → act → test → review → iterate cycle that runs until the goal is met or the token budget exhausts — promoted to an official stable “Goal mode” on May 21, 2026.

Layer 2: Orchestration and adversarial review. A first-order discovery of loop practice was the Goodhart failure: an agent that both develops and tests its own work learns to pass the tests rather than satisfy the specification. The response was adversarial separation — distinct developer and verifier agents, ideally with different priors. Anthropic’s Dynamic Workflows (research preview, June 2026) lets Claude write orchestration scripts on the fly, spawn coordinated sub-agent fleets, and validate results before presenting them; Jarred Sumner credited dynamic workflows and adversarial code review for enabling Bun’s port from Zig to Rust in six days. Cross-provider review institutionalized the pattern at the ecosystem level: a Codex plugin for Claude Code — documented in third-party practitioner writeups, though we find no confirmation in OpenAI’s official documentation that it is an OpenAI-maintained product — exists explicitly so that one vendor’s model writes code and another’s reviews it, countering the documented sycophancy bias of models grading work that resembles their own output.

Layer 3: Model-native self-verification. With Claude Fable 5 (released June 9, 2026, the first model in Anthropic’s Mythos class), verification moved inside the model: it writes its own tests, uses vision to check outputs against goals, sustains multi-day goal-directed runs with sub-agent delegation, and — at the highest effort setting — reflects on and validates its own work. GitLab’s early-tester language is precise on the economics: measurable improvements in *first-shot correctness on well-specified problems*, single-pass implementations of systems that previously took days of iteration, and agent workflows that previously required manual checkpoints now running to completion. GitHub’s internal benchmarks found Fable 5 completing equivalent autonomous work with fewer tool calls and lower token consumption than prior Opus-tier models.

How far did it get? Two numbers must be held simultaneously. SWE-bench Verified now reads 95–95.5% — the benchmark is approaching its ceiling, though the remaining ~5% (roughly 25 of 500 tasks) cannot be dismissed as statistical noise on any evidence we possess; more

consequentially, METR’s analysis found that about half of test-passing SWE-bench Verified PRs would likely not have been merged by the repositories’ actual maintainers, so even “solved” benchmark tasks overstate deployable completion. The realistic frontier evaluation, Frontier-Code (autonomous patches on real open-source repositories), shows Fable 5 leading with a **score** of 29.3 on the Diamond difficulty tier (the associated pass rate, reported separately, is roughly 30%), versus scores of 13.4 for Opus 4.8 and 5.7 for GPT-5.5. Read together: the self-contained loop has largely closed for well-specified benchmark tasks — with the maintainer-merge caveat attached — while frontier-difficulty work still fails roughly seven times in ten; the generation-over-generation jump (13.4 → 29.3 in score, a 2.2x gain) is the strongest available signal of how fast the frontier is moving. What self-verification has *not* solved is trust: Fable 5’s system card notes somewhat elevated vulnerability to prefill attacks and occasional reckless or destructive goal-directed actions (including guardrail bypassing and file deletion), and recommends human approval on irreversible operations. Capability closure and trust closure are different events; only the first has occurred.

2.3 The Cost Explosion

The vanguard’s methods were token-intensive by design, and by early 2026 token spend had become a labor-cost-scale line item. Tomasz Tunguz’s arithmetic became the reference frame: a 75th-percentile US software engineer at \$375,000 in salary plus \$100,000 in inference costs implies fully loaded personnel cost of \$475,000 — over 20% tokens. Anthropic quietly doubled its own estimate of token costs per engineer in April 2026. Microsoft’s Charles Lamanna reported engineering candidates explicitly negotiating token budgets as a fourth component of compensation alongside salary, bonus, and equity; internal “tokenmaxxing” leaderboards ran at Meta and OpenAI; one Ericsson engineer reported spending more on Claude than he earned in salary (employer-paid). The most fully documented case is Meta’s: an internal memo to roughly 6,000 employees warned that internal AI usage was on track for billions of dollars in 2026, disclosed that employees had consumed 73.7 trillion tokens in about thirty days as tracked on a leaderboard named “Claudeconomics,” and drew a pointed corrective from CTO Andrew Bosworth — that all motion is not progress and token usage is not a measure of impact of any kind. The leaderboard, by ranking staff on consumption volume, had inadvertently incentivized spend over output — the tokenmaxxing pathology in its purest institutional form.

The usage distribution stratified into three tiers. The consumer tier (~\$20/month) covers most developers — agentic tools are accessible but rationed. The power-user tier (\$100–200/month: Claude Max, Codex Pro) is where delegation replaces autocomplete — multiple parallel agent sessions, higher-abstraction direction. The frontier tier (\$1,000+/day) is exemplified by StrongDM’s “Software Factory,” a three-person team building security software via agent swarms under the rule of thumb that an engineer who has not spent \$1,000 on tokens today has room for improvement. We adopt a working definition of the *agentic user* as one whose combined subscription and API spend reaches at least the Max-equivalent \$100/month threshold — a level that cleanly separates workflow delegation from autocomplete, with the caveat that enterprise-seat API users must be counted separately.

Left ungoverned, this stratification produced the era’s cautionary tales: Uber exhausted its entire 2026 AI coding budget in four months and capped spending at \$1,500 per tool per month — even as nearly 95% of its engineers used AI tools monthly and close to 70% of committed code was AI-generated, with its COO stating plainly that the link between token spend and

measurable output “is not there yet”; Microsoft revoked most of its developers’ Claude Code licenses in May (individual engineers had been spending \$500–2,000/month, though the vendor-strategic motive of migrating developers to Microsoft’s own stack confounds a pure cost reading); one unnamed company ran up a \$500 million Claude bill in a single month with no caps or attribution; a healthcare enterprise consumed a trillion tokens — over \$6 million unplanned — in six months; and at least one CFO discovered a single engineer who had spent \$40,000 on tokens in thirty days.

2.4 Heterogeneous Orchestration Emerges

The cost pressure forced a decomposition of cost per completed task into three multiplicative levers:

Cost per completed task = $P \times T \times N$, where P is the price per token, T the tokens per loop iteration, and N the number of iterations.

This prices one successfully completed run. The expected cost of a completed *task* divides further by the probability q that a run completes ($P \times T \times N / q$), amortizing failed and abandoned runs over successes; for tractability we hold q aside and treat failure-loop costs qualitatively, which biases the stated per-task cost downward wherever failure rates are material.

Stage Two’s efficiency response attacked all three levers with heterogeneous multi-model orchestration. On P : routing each subtask to the cheapest model adequate for it arbitrages an extraordinarily steep capability-price curve — GPT-4-level performance fell from roughly \$60 to \$0.30–0.75 per million tokens (>98%) while frontier-tier models retained premium pricing. On T : prompt caching (cache reads at 0.1x base input price — a vendor-published rate, and the one unambiguously quantified efficiency lever in this decomposition), context compaction, and batch processing. On N : see Part III.

Orchestration institutionalized at three levels. Vendors themselves became routers — OpenAI’s official model documentation positions GPT-5.4 mini for narrow sub-agent execution, and practitioner accounts describe Codex workflows in which the full model handles planning and final judgment while the mini variant executes; the specific planner/worker/judge division, however, is practitioner-documented rather than an officially specified Codex policy. Cross-provider review became ecosystem practice (the third-party-documented Codex plugin for Claude Code). And a third-party orchestration layer formed — Zen MCP coordinating 50+ models across providers for multi-model reviews and pipelines; GitHub’s Agent HQ as a permissioned mission control over heterogeneous agents; routing heuristics (“cheap loops to Codex-class, expensive-retry long-horizon work to Claude premium”) becoming standard practitioner guidance. The practitioner ethos of the period was captured in a widely shared formulation: after prompt engineering and context engineering, the discipline is now simply *compute engineering* — deciding how many model-tokens a given task deserves (“code review? \$10/PR via three parallel Opus subagents; internal brief? route it to a nano model”).

Three caveats temper the orchestration thesis. Frontier models share large fractions of training data, so cross-model review cannot catch correlated blind spots (the N-version-programming gain is bounded by error independence). No controlled study yet quantifies the defect-detection advantage of cross-model over same-model review; the evidence is mechanism plus practitioner reports. And vendor incentives push against heterogeneity — each

lab prefers vertical lock-in, making the contest for the router seat (vendor apps vs. GitHub vs. Cursor vs. independent layers) a defining industrial battle.

2.5 Reinterpreting the Revenue Hypergrowth

How much of the \$9B → \$47B run-rate explosion was durable demand? Stage Two's correct accounting requires two corrections.

The extensive margin is largely saturated. Product-level adoption figures (Claude Code at 18% of developers globally, 24% in the US and Canada, as of January 2026) mislead if read as market headroom: across all products — Claude Code, Codex (whose weekly active users went 2M → 3M → 4M between March and late April 2026), Cursor, Copilot (58% any-use), Antigravity, Replit and others — roughly 90% of professional developers worldwide report regular use of AI coding tools (JetBrains, January 2026), and fewer than one developer in ten uses none. Regular use, however, is not daily use: only 51% of professional developers reach daily frequency (Stack Overflow, 2025). The correct model therefore has a *nearly fixed* user base N and a single forecast-relevant object: the distribution of **delegation intensity** I (tokens of verified work delegated per developer per period), with total tokens = $N \cdot E[I]$. All meaningful growth is intensive-margin growth — including the migration from occasional toward daily use that the 90%-regular/51%-daily gap itself measures: the migration of the distribution's body toward its tail, plus movement of the tail itself.

The concentration arithmetic. The intensity distribution is heavily right-skewed (Jellyfish telemetry: heavy users consume ~10x the tokens of average users), which makes aggregate outcomes acutely sensitive to concentration. Consider a firm whose top decile of users consumes share w of tokens, and suppose rationalization halves the tail's usage while the remaining 90% of users triple theirs. Total usage changes by a factor of $0.5w + 3(1 - w)$, which equals 1 at $w = 0.8$. Above 80% concentration, tail compression mechanically dominates and total tokens *fall* (at $w = 0.9$, to 75% of the prior level); below it, body expansion wins (at $w = 0.5$, total rises 75%). Where real-world concentration sits relative to this threshold is unmeasured, and two reference points bracket it loosely. Jellyfish's multiple, read against the remaining users, implies a top-decile share near 53% in a governed organization. The spreads reported in ungoverned ones — a single engineer at \$40,000 against typical users at \$500–2,000 — point considerably higher, plausibly to 0.8–0.9, though anecdotes of that kind cannot fix a share without user counts and the full distribution (Appendix A). The implication is conditional but sharp: firm-level token totals will *diverge* during rationalization, falling where concentration exceeds the threshold and rising where it does not, which makes the top-decile token share within enterprises the single most important quantity to track in H2 2026.

The corollary for revenue interpretation: a material fraction of Stage Two's hypergrowth was tuition — failed loops, uncached context resends, leaderboard-driven tokenmaxxing. Its size is not observable from outside: no public dataset separates productive from unproductive tokens at the vendor level. Its existence is not in doubt. Bosworth's corrective that token usage measures nothing, Uber's COO stating the spend-to-output link is absent, the \$500 million uncapped monthly bill, Vaudit's audit of disputed charges for failed requests and retry loops all describe it directly — and Coinbase (\$3.2) halved its AI spend while *increasing* token volume, which is possible only if the prior configuration was buying tokens that produced nothing. That fraction, whatever its size, is scheduled for clawback in Stage Three. The coming deceleration

in vendor run-rates should therefore be read, at least in part, as revenue-quality normalization rather than demand destruction — a distinction markets will likely fail to draw in real time.

Part III. The Maturation Era

The Metered Equilibrium

3.1 The Demand-Side Reckoning

If the dominant constraint of H1 2026 was supply (compute scarcity, rate limits, infrastructure strain), the dominant constraint of H2 2026 is demand discipline. The June 2026 reporting cycle marks the turn: TechCrunch’s “the token bill comes due” synthesis (Uber’s exhausted annual budget, Microsoft’s license revocations, Priceline’s 4–5x Cursor renewal) lands alongside Forrester’s prediction that enterprises will defer 25% of planned AI spend into 2027 as financial scrutiny rises, with fewer than one-third of decision-makers able to tie AI investments to specific financial outcomes. Deloitte published a CFO guide to AI token economics in April 2026 — a topic that did not exist on finance radars eighteen months earlier. Enterprise AI budgets now face renewal reviews under a higher bar: productivity gains that are real but diffuse — the Stage One TFP problem reappearing at the firm level — no longer justify an uncapped line item.

The crucial question is *where on the intensity distribution the constraint binds*, and here the single most consequential measurement of the period is Jellyfish’s: heavy token users were about 2x more productive while spending 10x the tokens, so the best marginal ROI lies in moving *average* users to *moderate* usage, not in pushing power users higher. Marginal token productivity is declining in the tail and still high in the body. The rational CFO optimization is therefore asymmetric — trim the unmetered tail, fund the body’s migration — which means enterprise discipline does not switch off the principal growth engine identified in §2.5; it institutionalizes it, redirecting subsidy from leaderboard excess to governed adoption. Industry participants describe the correction in exactly these terms (a “healthy swing” away from token-burning culture) — the migration from *tokenmaxxing* to *valuemaxxing*: token spend governed by measured value rather than by volume.

The closest historical precedent is explicit in contemporary analysis: the AWS bill horror stories of 2012–2015, which produced cloud FinOps as a discipline. The infrastructure is repeating itself — a Linux Foundation standards body for token usage and billing metrics launches in July 2026, and a vendor ecosystem (Pay-i, Paid, Jellyfish, Waydev, Faros AI) is racing to give enterprises cost attribution and ROI proof. The cloud lesson we rely on is qualitative and does not require a disputed statistic: FinOps arrived as a response to bill shock, and its arrival coincided not with the end of cloud spending growth but with the promotion of cloud from experiment to governed operating expense. We are aware of no year in which aggregate US cloud spending declined. The mechanism worth carrying over is that one — governance as a precondition of durable budget rather than a brake on it — and it is a hypothesis about token spend, not a demonstrated parallel. The disanalogy must also be stated: cloud ROI had a clean counterfactual (server costs); AI coding ROI inherits the diffuse-gains measurement problem.

If the measurement layer fails to convert diffuse gains into attributable ones before the 2027 renewal cycle, the median segment faces real cuts — the principal bear case of this report.

3.2 Dollars vs. Tokens: The Decoupling

A decisive analytical distinction: **CFO constraints bind on dollars, not on tokens**. Per-token prices for fixed capability have fallen >98% since 2022 even as enterprise bills rose to labor-cost scale (§2.3) — and the discipline phase itself accelerates routing toward cheaper models. Under a *frozen* dollar budget, continued price deflation plus mix-shift toward routed, cached, batch-discounted consumption plausibly buys 2–5x more tokens per year. The decoupling is no longer merely theoretical: Coinbase, having shifted much of its workload to lower-cost models under automated routing that selects on task complexity, price, and caching efficiency, reported cutting AI spending roughly in half *while token usage rose* — dollars and volume moving in opposite directions within a single firm. The demand-side constraint therefore hits dollar spending growth directly and token volume growth only through a buffer. What it hits hardest is **vendor revenue quality**: mix degradation from premium tiers toward routed blends, margin pressure, and run-rate deceleration that — per §2.5 — partially reflects the clawback of Stage Two tuition. Separating “deceleration from rationalization” from “deceleration from demand saturation” will be the central analytical task of H2 2026; the dollar/token decoupling is its sharpest diagnostic.

3.3 The Efficiency Frontier: Loop Collapse

Within the $P \times T \times N$ decomposition, the largest remaining headroom is **N — the loop count**. P (price at fixed capability) declines on a fast but anticipated slope; T (tokens per iteration) yields a largely one-time gain from caching and compaction, bounded by the published 0.1x cache-read rate. N, by contrast, collapses in generational steps rather than continuously: GitLab reports Fable 5 delivering single-pass implementations of systems that previously took days of iteration, and GitHub reports equivalent work completed with fewer tool calls. These are vendor and early-tester accounts; no published series yet tracks average loops-per-task across model generations, which is why that series heads the efficiency indicators of §4.1.

The mechanism is **search internalization**, structurally identical to the policy-network/rollout trade-off in game-playing systems: a weak policy requires massive external search (the Ralph loop’s hundreds of brute-force retries are exactly this), and a stronger policy achieves the same solution quality in a fraction of the rollouts. The convergence of pass@k toward pass@1 is search moving from the harness into the weights. Two auxiliary channels compound it: persistent memory (Beads, Hindsight-class systems) eliminating per-iteration re-exploration of the codebase — the Ralph loop’s “fresh context each iteration” was, in hindsight, a deliberate inefficiency — and specification quality reducing the agent’s wander radius (§3.4). Combined, these channels drive cost per completed task *for a fixed task class* down faster than per-token price alone. The mechanism identifies the direction firmly; the rate remains unmeasured. What the argument below requires is only that the decline is steep enough to make previously uneconomic tasks economic — which Fable 5’s multi-day runs on formerly infeasible work attest.

Two equilibrating mechanisms prevent this collapse from translating into proportional token decline. First, **task-frontier reallocation**: N-savings on solved task classes are immediately redeployed against the FrontierCode-style 70% that previously failed, and the marginal task

always sits at the loop-budget frontier — when easy work drops from 500 loops to 5, the budget moves to work that formerly failed at 500. Fable 5 is the live demonstration: fewer tokens per task, yet multi-day runs on previously infeasible work, and Anthropic’s per-engineer token estimate *doubled* in the very month efficiency research matured. The Jevons paradox, recurring at the loop level. Second, **per-task pricing capture**: because N-reduction is embodied in the model, vendors price it — premium per-token rates are justified precisely by fewer loops, and practitioners already quote informal cost-per-feature figures — though no vendor rate card yet formalizes per-task pricing, which is why its appearance there is a first-order monitoring item (§4.2). The market-clearing price converges on cost per completed task, and the surplus from loop collapse is split between customer savings and vendor margin according to competitive intensity. This is also why loop efficiency is not a pure deflation shock to vendor revenue: it can be absorbed as a pricing-regime migration from token metering toward task metering.

One asymmetric risk deserves registration: P and T decline continuously, but N collapses in generational steps. A generation in which N collapses faster than the task frontier expands would produce a quarter or two of genuine token-volume decline — and if it coincides with the rationalization level-shift of §3.1, the combined deceleration could look considerably deeper than its fundamentals.

3.4 The Specification Engine

The 70% FrontierCode failure rate conflates two distinct failure modes: capability-bound failures (the model cannot solve a well-specified problem) and **specification-bound failures** (intent was never closed, so the agent converges on the wrong target). Evidence suggests the second mode is large. Contemporary readings of Anthropic’s delegation-gap finding are explicit that the gap is context, not capability — current models can handle far more than 20% of tasks autonomously when the agent possesses the codebase knowledge, constraints, stakeholders, and failure modes the task assumes — and engineers report withdrawing delegation precisely when tasks become design-heavy or ambiguous, i.e., when they become unspecifiable, not undoable. GitLab’s careful phrasing of Fable 5’s first-shot gains — *on well-specified problems* — marks specification as the explicit conditional.

The delegable frontier therefore expands along **two independent axes — model capability and specification quality** — and Stage Three’s underappreciated development is that the second axis acquired its own engine, improving from both sides simultaneously.

On the tool side, specification elicitation has itself been agentized. The exemplar is Matt Pocock’s grill-me skill (13,000+ upvotes of community discussion; portable across Claude Code, Codex, and Cursor), whose interview discipline inverts the roles: the human gives a deliberately vague two-sentence description, and the agent interrogates — one question per turn, each accompanied by a recommended answer, exploring the codebase first whenever exploration can resolve a question, walking the design tree depth-first until shared understanding is reached. Each question costs the human 5–10 seconds; a full interrogation runs about ten minutes. Two design details carry the economics: recommended answers mean the human corrects a prior rather than writing on a blank page, and codebase-first exploration converts organizational tacit knowledge (existing conventions, similar patterns) into specification automatically. The cost of producing a closed spec has collapsed from hours of PRD-writing to minutes of guided correction. Comparative evaluations of the planning-tool category (Plan

Mode vs. grill-me vs. Superpowers) now begin from the shared premise that generated-code quality starts with the plan, not the prompt. On the human side, the interrogation itself is training: answering thirty-seven forced-choice design questions per feature is learning-by-doing in specification discipline, and it is precisely this discipline — not raw prompting — that constitutes the heavy users’ most valuable diffusible knowledge. The combined effect de-skills the developer-to-agent-PM transition that Stage Two’s vanguard achieved through expensive trial-and-error, lowering the friction coefficient on the body migration that drives Stage Three growth.

Token composition migrates upstream accordingly: implementation-loop tokens shrink with N-collapse while a new specification-token category (interview dialogues, pre-exploration, adversarial spec critique) grows — and it is the highest-marginal-product token category in the stack, since a ten-minute interview (tens of thousands of tokens) pre-empts dozens of wandering loops (millions). Rational CFO optimization reallocates from implementation tokens to specification and verification tokens. Early telemetry already shows the shift: an empirical study of a multi-agent development framework found the code-review stage consuming a large share of total tokens while the initial coding phase averaged a small share — verification, not generation, now dominates the bill. Whether hierarchical routing — frontier model as lead orchestrator, budget models as workers — preserves accuracy at materially lower cost would determine how cheaply this reallocation can be executed; the question remains open, with no controlled evaluation published. The case for the reallocation does not depend on it, resting instead on the asymmetry of marginal products already stated: the interview is cheap, the loops it pre-empts are not.

The asymptote must be stated. An irreducible core resists specification: preference-revelation tasks where the human cannot know what they want until they see an artifact (for these, the interactive loop *is* the specification process; elicitation shortens but cannot delete it); organizationally political specifications, where “done” is a stakeholder negotiation rather than a technical predicate; and spec-level Goodhart, where over-formalized acceptance criteria invite agents to satisfy the letter while missing the intent — the “learns to pass the tests” failure relocated upstream. The frontier expands toward this asymptote without reaching it; the operative point is that from a Diamond-tier score of 29.3, the distance is long.

3.5 The Forecast

The synthesis is a revenue-growth identity:

Coding-agent revenue growth $\approx$ growth in demand for completed tasks – customer-captured decline in price per task

where task demand growth = (body migration of the intensity distribution) $\times$ (task-frontier expansion), and the price-per-task decline is the customer-captured share of P, T, and N improvements. The responsiveness of demand to falling prices is the decisive term, and it has not been measured. Aggregate spend grew enormously — Anthropic’s run-rate up roughly 47x in the year and a half to mid-May 2026 — across a technology cycle in which capability-adjusted prices fell more than 98%. The two series do not form a demand curve: one is a single firm’s revenue, the other an economy-wide quality-adjusted price index, and between them sit new-customer acquisition, quality improvement, product-mix shifts, and supply expansion. What

they establish is a pattern consistent with demand elastic enough that falling prices have so far expanded total spend rather than shrinking it. We proceed on that hypothesis, which the \$4.1 dashboard is built to test. If it holds, efficiency — loop collapse and the specification engine alike — is the *diffusion mechanism*, not the brake: what converts a median developer into a power user is not exhortation but the task that cost \$50 costing \$5.

Scenario tree (probability-weighted):

Scenario	Weight	H2 2026	2027
Base: rationalize-then-reaccelerate	55–60%	Dollar growth decelerates sharply (the roughly eight-week doubling ends; the Forrester 25% deferral and firm-level rationalization level-shifts land); token volume growth decelerates far less, buffered by price deflation and routing mix	Measurement infrastructure matures, deferred spend executes, compute supply arrives; growth resumes on cleaner unit economics — the cloud-FinOps path
Bear: measurement-failure air pocket	20–25%	As base	Diffuse gains remain unattributable through the renewal cycle; median-segment budgets cut in real terms; extended plateau. Even here, verified-ROI frontier workflows and routed low-cost volume keep aggregate tokens from absolute decline
Bull: per-task ROI closes early	15–20%	Fable-class autonomy makes per-completed-task pricing and attribution explicit, shortening the discipline phase to a quarter or two	Re-acceleration pulls forward

Saturation verdict. Total token consumption does not saturate within the forecast horizon. Saturation requires the entire intensity distribution to converge on a *static* ceiling; instead the body is migrating toward the current ceiling (the diffusion of vanguard practice, now tool-scaffolded), and the ceiling itself is being pushed upward by compute supply (the gigawatt-scale capacity contracted for late 2026–2027) and by autonomy gains that raise one human’s span of control over parallel agents. The growth identity is governed by the task-frontier expansion rate — FrontierCode-class scores climbing from Fable 5’s Diamond-tier 29.3, multiplied by the specification engine — interacting with a demand response so far consistent with elasticity above unity (above). The binding constraints, in order of arrival: compute supply (now), ROI attribution (2027 renewal cycle), and only in the long run the marginal economic value of additional verified software — a constraint nowhere near binding.

Part IV. Monitoring and Implications

4.1 The Indicator Dashboard

The framework above reduces to a compact set of falsifiable indicators, listed with the hypothesis each one tests.

Distribution and demand indicators. The full-delegation rate (Anthropic’s 0–20% figure) is the single most authoritative measure of body migration; movement into the 40–60% range would mark the threshold crossing from complementarity to substitution. The median-to-mean ratio of per-developer token spend measures whether the body is catching up to the tail (a ratio approaching 1 is the precondition for any genuine saturation discussion). Within-firm top-decile token share is the rationalization diagnostic: a fall from the high concentrations plausibly present in ungoverned firms (suggested, though not yet measured, to lie in the 80–90% range; §2.5) toward 50–60% confirms the trim-the-tail-fund-the-body dynamic rather than demand destruction; concentration stabilizing while totals stagnate would be the first true saturation signal.

Decoupling and discipline indicators. The spread between vendor dollar-revenue growth and token-volume growth (the §3.2 decoupling) distinguishes rationalization from saturation in real time. Rate-limit binding frequency tests whether the tail remains supply-rationed. Execution versus re-deferral of Forrester’s 25% deferred spend in 2027 budgets adjudicates between the base and bear scenarios, as does the adoption velocity of the Linux Foundation token-metrics standards and the appearance of cost-per-merged-PR-class metrics in earnings calls.

Efficiency and frontier indicators. Published pass@1 trajectories and average loops-per-task from vendor telemetry track N-collapse. The emergence of per-task (rather than per-token) pricing in vendor rate cards signals surplus capture migrating to the model layer — a partial reversal of the commoditization thesis worth flagging the moment it appears. Spec-artifact density (PRD/ADR files per active repository, approximable from public GitHub data) and the share of sessions initiated in plan/interview modes track the specification engine. Frontier-Code-class scores and pass rates remain the cleanest measure of the capability axis. Finally, H1 2027 TFP data is the ultimate arbiter: the first appearance of an efficiency-channel acceleration would mark the macro arrival of Stage Three.

4.2 Where the Margin Migrates

The strategic conclusion follows the report’s production logic. As model capability commoditizes along the 98% deflation curve and verification automates, the scarce resource becomes *token efficiency per verified unit of output* — and the layer that controls it captures the margin. That layer is the control plane of the token economy: routing (which model, which task, which budget), measurement (cost and ROI attribution per completed task), and governance (caps, policies, audit). Token FinOps is crystallizing as a category in real time — a standards body, a dedicated vendor cohort, token spend installed as a board-level engineering KPI alongside headcount and cloud cost. The vendor formation accelerated visibly through mid-2026: bill-audit entrants (Vaudit’s TokenAudit, launched June 30, reported finding roughly \$1.7 million in disputed charges — failed requests, retry loops, model-pricing discrepancies — across \$34 million of Anthropic and OpenAI invoices reviewed for 60 companies, though both vendors contested the findings), engineering-ROI platforms (Journi’s DevOS, Exceeds’ commit-level

attestation), and Big Four frameworks (EY’s *Total Cost of Agents* series formalizing “agentic FinOps”). The pattern recapitulates the post-2012 emergence of cloud FinOps, compressed into quarters rather than years. The contest for the router seat — vendor-vertical (Codex’s internal model router), platform (GitHub Agent HQ), harness (Cursor), and independent (OpenRouter, Zen-class orchestration) — is the defining industrial structure question of Stage Three. The one development that would partially reverse the model-commoditization thesis is vendor success in per-task pricing, which would let the model layer recapture loop-efficiency surplus; its appearance on rate cards is therefore a first-order monitoring item.

4.3 Risks to the Thesis

Four risks, in descending order of our concern. **Measurement failure** (the bear scenario): if diffuse productivity cannot be converted into attributable ROI before the 2027 renewal cycle, median-segment budgets face real cuts and the maturation era begins with contraction rather than governed growth. **Run-rate credibility**: the revenue figures anchoring Stage Two’s narrative are vendor-disclosed annualized snapshots with documented internal tensions; quarterly realized revenue should be the cross-check of record. **Compute timing**: contracted capacity arriving late pushes the supply constraint deeper into 2027, suppressing the very tail whose practices seed the body’s migration. **Correlated blind spots**: the verification stack’s adversarial layer presumes error independence across models that shared training corpora may not deliver; a high-profile correlated failure in production would re-tighten human verification requirements and partially restore the $\sigma = 0.25$ regime the entire Stage Three thesis assumes is dissolving.

Appendices

Appendix A — The 80% Concentration Threshold: Derivation

Let the top decile of users consume share w of a firm’s tokens, the remainder $1 - w$. A rationalization policy that scales tail usage by factor a and body usage by factor b changes total usage by $\Delta = aw + b(1 - w)$. For the canonical policy discussed in the text ($a = 0.5$, $b = 3$), $\Delta = 1$ at $w = 0.8$: total usage falls if and only if pre-rationalization concentration exceeds 80%. At $w = 0.9$, $\Delta = 0.75$ (a 25% decline); at $w = 0.5$, $\Delta = 1.75$ (a 75% rise). Generally, the breakeven is $w^* = (b - 1) / (b - a)$; the qualitative conclusion — aggregate outcomes are governed by concentration, and firm-level totals diverge during rationalization — is robust to the choice of (a, b) . Two reference points frame the empirics. A heavy-user multiple of 10x *relative to the remaining users* over a 10% population implies $w \approx 0.53$; the multiple must be read relative to the complement, since a top decile at 10x the *overall* average would account for the entire total. The spreads reported in ungoverned organizations (\$40,000 against \$500–2,000 monthly) point higher, plausibly to 0.8–0.9, but identifying w from them would require user counts and the full spending distribution. Direct measurement of within-firm top-decile share is accordingly the first-order data request of the \$4.1 dashboard.

Appendix B — Data Conventions and Known Disputes

Run-rate revenue (“ARR”) throughout refers to vendor-disclosed annualized snapshots (most recent month $\times$ 12), not audited revenue. Known tensions: Anthropic’s disclosed Q1 2026 revenue of \$4.8 billion and Q2 guidance of \$10.9 billion are difficult to reconcile with the full prior sequence of ARR announcements (Zitron, 2026); we treat the trajectory as directionally reliable and the levels as upper bounds. Benchmark figures dated June 9–10, 2026 derive from the Claude Fable 5 / Mythos 5 system card and third-party analyses thereof; Fable 5 figures (not Mythos 5) are quoted wherever the distinction matters, as the latter is restricted to vetted partners. Survey adoption figures mix instruments (GitHub 2023, $n = 500$ US enterprise developers, ever-use; Stack Overflow 2025, $n \approx 49,000$, daily-use among professionals; JetBrains January 2026, regular-use worldwide; Sonar 2026; Harness 2026) whose sampling frames and question wordings differ materially. Ever-use, regular-use, and daily-use rates are not interchangeable, and we cite each with its source rather than averaging across instruments. The NBER elasticity dating (Copilot autocomplete variation, June 21 – December 26, 2022) follows the paper’s identification design; the attenuation estimates use the full multi-generation panel.

Appendix C — Glossary

Agentic user: a developer whose combined subscription and API spend reaches at least \$100/month (Max-equivalent), the threshold separating workflow delegation from autocomplete usage. **Delegation gap:** the spread between AI usage share (~60% of work) and full-delegation share (0–20% of tasks) documented in Anthropic’s 2026 Agentic Coding Trends Report. **FrontierCode:** a frontier-difficulty agentic coding evaluation based on autonomous patches to real open-source repositories; the Diamond tier is its hardest subset. **grill-me:** an agent skill (Matt Pocock) in which the agent interviews the developer one question at a time, with recommended answers and codebase pre-exploration, until a specification is closed. **Ralph loop (Ralph Wiggum technique):** running a coding agent in an automated loop against a fixed specification until it is satisfied (Geoffrey Huntley, 2025); the origin of productized goal modes. **Tokenmaxxing:** ungoverned maximization of token consumption — defaulting to the most capable model for every task without routing or cost visibility and, in its purest institutional form, treating consumption volume itself as a performance signal (internal leaderboards). **Valuemaxxing:** the Stage Three counter-ethos to tokenmaxxing — token spend governed by measured value per token rather than by consumption volume. **Weak-link production:** a production function in which output is determined by the scarcest complementary input; with elasticity of substitution $\sigma = 0.25$, AI and human effort are strong complements and human verification binds.

References

- Anthropic. *2026 Agentic Coding Trends Report*. 2026. <https://resources.anthropic.com/hubfs/2026%20Agentic%20Coding%20Trends%20Report.pdf>
- Anthropic. “Claude Fable 5 and Claude Mythos 5.” June 9, 2026. <https://www.anthropic.com/news/claude-fable-5-mythos-5>
- Anthropic. “Claude Fable.” Product page. June 2026. <https://www.anthropic.com/claude/fable>

Anthropic. “Anthropic raises \$65B in Series H funding at \$965B post-money valuation.” May 28, 2026. <https://www.anthropic.com/news/series-h>

Azure (Microsoft). “Claude Fable 5 available today in Microsoft Foundry: Powering the next era of autonomous agents.” June 9, 2026. <https://azure.microsoft.com/en-us/blog/claude-fable-5-is-now-available-in-microsoft-foundry-powering-the-next-era-of-autonomous-agents/>

Businesswire (Forrester). “Forrester’s 2026 Technology & Security Predictions: As AI’s Hype Fades, Enterprises Will Defer 25% of Planned AI Spend to 2027.” October 2025. <https://www.businesswire.com/news/home/20251028641086/en/Forresters-2026-Technology-Security-Predictions-As-AIs-Hype-Fades-Enterprises-Will-Defer-25-of-Planned-AI-Spend-to-2027> (The 25% deferral is a forecast, not a realized figure.)

Business Model Analyst. “AI Token Bills Explode and Companies Scramble to Regain Control.” June 2026. <https://businessmodelanalyst.com/ai-token-costs-tokenomics-foundation-enterprise-spending/>

BizTech Magazine (Eddy, N.). “AI Tokenomics: How Token-Based Pricing Is Reshaping Enterprise AI Strategy.” July 2026. <https://biztechmagazine.com/article/2026/07/ai-tokenomics-how-token-based-pricing-reshaping-enterprise-ai-strategy-perfcon>

Bureau of Labor Statistics. “Total Factor Productivity — 2025.” News release, March 19, 2026. https://www.bls.gov/news.release/archives/prod3_03192026.htm

Bureau of Labor Statistics. “Productivity and Costs, First Quarter 2026.” News release, June 4, 2026. https://www.bls.gov/news.release/archives/prod2_06042026.pdf

codecentric. “The Ralph Wiggum Loop: Autonomous Code Generation with a Fresh Context.” April 2026. <https://www.codecentric.de/en/knowledge-hub/blog/the-ralph-wiggum-loop-autonomous-code-generation-with-a-fresh-context>

Daily.dev. “Vibe Coding in 2026: How It Works and When to Use It.” May 2026. <https://daily.dev/blog/vibe-coding-how-ai-changing-developers-code/>

Demirer, M., Musolff, L., & Yang, S. *Writing Code vs. Shipping Code: Productivity Effects Across Generations of AI Coding Tools*. NBER Working Paper No. 35275, 2026. <https://www.nber.org/papers/w35275>

EY. *The Total Cost of Agents* (agentic FinOps whitepaper series, first edition). 2026. https://www.ey.com/en_us/insights/ai/agentic-ai-token-costs

Exceeds AI. “6 Strategies for Managing AI Coding Token Costs in 2026.” June 2026. <https://blog.exceeds.ai/ai-coding-token-costs-2026/>

Dev Journal (earezki). “Vibe Coding Audit Failure: 96% of Developers Distrust AI-Generated Code.” April 2026. <https://earezki.com/ai-news/2026-04-26-vibe-coding-just-failed-its-first-real-audit/>

DEV Community (javatarz). “Multi-Agent Development Workflows with Claude Code.” May 2026. <https://dev.to/javatarz/multi-agent-development-workflows-with-claude-code-n23>

Digital Applied. “Claude Fable 5 & Mythos 5: The Frontier, Split in Two.” June 9, 2026. <https://www.digitalapplied.com/blog/claude-fable-5-mythos-5-release-benchmarks-2026>

Digital Applied. “Claude Fable 5 & Mythos 5: Agentic Coding Deep Dive.” June 9, 2026. <https://www.digitalapplied.com/blog/claude-fable-5-mythos-5-agentic-coding-deep-dive-2026>

Digital Applied. “Claude Code vs Codex vs Jules: Q2 2026 Benchmark Matrix.” April 2026. <https://www.digitalapplied.com/blog/claude-code-vs-codex-vs-jules-q2-2026-matrix>

elevatex. “Token Spend: The New Engineering KPI for CTOs in 2026.” May 2026. <https://elevatex.de/blog/ai/token-spend-engineering-kpi/>

elvex. “AI Token Cost Enterprise: Stop Budget Blowouts in 2026.” May 2026. <https://www.elvex.com/blog/ai-token-cost-enterprise-budget-control>

Faros AI. Engineering telemetry across 1,255 teams (PR review time +91%, merged PRs +98%). As reported in Addy Osmani, “Code Review in the Age of AI,” January 2026.

FutureSearch. “Anthropic Revenue and Valuation in 2026 Leading to IPO” (financial forecast; ARR walk). June 2026. <https://futuresearch.ai/anthropic-financial-forecast/>

GitClear. *AI Copilot Code Quality: 2025 Data Suggests 4x Growth in Code Clones*. February 2025. https://www.gitclear.com/ai_assistant_code_quality_2025_research (211 million changed lines, January 2020 – December 2024; full report PDF: <https://gitclear-public.s3.us-west-2.amazonaws.com/GitClear-AI-Copilot-Code-Quality-2025.pdf>)

GitHub. “Survey reveals AI’s impact on the developer experience.” June 2023. <https://github.blog/news-insights/research/survey-reveals-ais-impact-on-the-developer-experience/> (n = 500 US enterprise developers; measures ever-use at work or personally.)

GitHub. “Claude Fable 5 is generally available for GitHub Copilot.” GitHub Changelog, June 9, 2026. <https://github.blog/changelog/2026-06-09-claude-fable-5-is-generally-available-for-github-copilot/>

GitLab. “Mythos-class Claude Fable 5 arrives on GitLab Duo Agent Platform.” June 9, 2026. <https://about.gitlab.com/blog/mythos-class-claude-fable-5-on-gitlab/>

Harness. *State of Software Delivery*. 2026. <https://www.harness.io/state-of-software-delivery>

Huntley, G. The Ralph Wiggum technique (canonical blog post, July 2025), as documented in the awesome-ralph compendium. <https://github.com/snwfdhmp/awesome-ralph>

Imas, A. “What Is the Impact of AI on Productivity?” March 2026. <https://aleximas.substack.com/p/what-is-the-impact-of-ai-on-productivity> (Survey of the productivity literature; source of record here for Brynjolfsson’s Financial Times argument on labor-output decoupling and for Furman’s concurrence with it.)

InfoQ. “Claude Code Adds Dynamic Workflows for Parallel Agent Coordination.” June 2026. <https://www.infoq.com/news/2026/06/dynamic-workflows-claude-code/>

JetBrains. “Which AI Coding Tools Do Developers Actually Use at Work?” (AI Pulse; *State of Developer Ecosystem* follow-up survey, January 2026). April 2026. <https://blog.jetbrains.com/research/2026/04/which-ai-coding-tools-do-developers-actually-use-at-work/>

Leanware. “Ralph Wiggum AI Agents: The Coding Loop of 2026.” January 2026. <https://www.leanware.co/insights/ralph-wiggum-ai-coding>

METR. “Measuring the Impact of Early-2025 AI on Experienced Open-Source Developer Productivity” (RCT). July 10, 2025. <https://metr.org/blog/2025-07-10-early-2025-ai-experienced-os-dev-study/>

METR. “Uplift Update: Revisiting the Early-2025 Developer Productivity RCT.” February 24, 2026. <https://metr.org/blog/2026-02-24-uplift-update/>

METR. “Many SWE-bench-Verified-Passing PRs Would Not Be Merged Into Main.” Research note, March 10, 2026. <https://metr.org/notes/2026-03-10-many-swe-bench-passing-prs-would-not-be-merged-into-main/>

MindStudio. “What Is the OpenAI Codex Plugin for Claude Code? How Cross-Provider AI Review Works.” 2026. <https://www.mindstudio.ai/blog/openai-codex-plugin-claude-code-cross-provider-review>

MindStudio. “Anthropic ARR Doubled Every 6 Weeks in 2026.” May 2026. <https://www.mindstudio.ai/blog/anthropic-arr-growth-9b-to-44b-2026>

MarketingProfs. “AI Update, July 3, 2026: AI News and Views From the Past Week” (Coinbase model routing; Vaudit billing audit). July 3, 2026. <https://www.marketingprofs.com/opinions/2026/55197/ai-update-july-3-2026-ai-news-and-views-from-the-past-week>

MLQ News. “Meta Caps Internal AI Token Spending After Costs Approach Billions in 2026.” June 2026. <https://mlq.ai/news/meta-caps-internal-ai-token-spending-after-costs-approach-billions-in-2026/>

OpenAI. “GPT-5.4 mini.” Official model documentation. <https://developers.openai.com/api/docs/models/gpt-5.4-mini>

OpenAI. “Long-running work in Codex.” Official documentation. <https://learn.chatgpt.com/docs/long-running-work>

OpenAI. Codex CLI release notes, v0.128.0 (April 30, 2026) through v0.133.0 (May 2026). <https://github.com/openai/codex/releases> (v0.128.0 introduced persisted /goal workflows with app-server APIs, model tools, runtime continuation, and TUI create/pause/resume/clear controls; Goal Mode was promoted to standard in the May 21, 2026 release wave.)

Morphllm. “AI Coding Costs (2026): Claude vs Codex vs Gemini, Real Monthly Spend From Token Math.” June 2026. <https://www.morphllm.com/ai-coding-costs>

O’Reilly, T. “Steve Yegge Wants You to Stop Looking at Your Code.” O’Reilly Radar, March 12, 2026. <https://www.oreilly.com/radar/steve-yegge-wants-you-to-stop-looking-at-your-code/>

Onoufrios (Substack). “The Emerging Token Gap: How AI Spending Is Splitting Engineers Into Tiers.” 2026. <https://onoufrios.substack.com/p/the-emerging-token-gap-how-ai-spending>

Osmani, A. “Code Review in the Age of AI.” January 2026. <https://addyosmani.com/blog/code-review-ai/>

Osmani, A. “The Code Agent Orchestra — what makes multi-agent coding work.” March 2026. <https://addyosmani.com/blog/code-agent-orchestra/>

Pocock, M. grill-me (MIT-licensed agent skill), with derivative documentation. <https://alirezarezvani.github.io/claude-skills/skills/engineering/grill-me/>

Rusin, A. “Claude Code Planning Tools Compared: Plan Mode vs Grill Me vs Superpowers.” May 2026. <https://blog.alexrusin.com/claude-code-planning-tools-plan-mode-vs-grill-me-vs-superpowers/>

Sacra. “Anthropic revenue, valuation & funding.” May 2026. <https://sacra.com/c/anthropic/SaaStr>. “Anthropic Just Hit \$14 Billion in ARR. Up From \$1 Billion Just 14 Months Ago.” February 2026. <https://www.saastr.com/anthropic-just-hit-14-billion-in-arr-up-from-1-billion-just-14-months-ago/>

Shipyard. “Multi-agent orchestration for Claude Code in 2026.” March 2026. <https://shipyard.build/blog/claude-code-multi-agent/>

Sonar. “Sonar Data Reveals Critical Verification Gap in AI Coding.” Press release, 2026. <https://www.sonarsource.com/company/press-releases/sonar-data-reveals-critical-verification-gap-in-ai-coding/>

Stack Overflow. *2025 Developer Survey* (n ≈ 49,000), AI section. <https://survey.stackoverflow.co/2025/ai>

StartupHub.ai. “Codex Now Tackles Long Tasks With New ‘Goals’ Feature.” May 2026. <https://www.startuphub.ai/ai-news/artificial-intelligence/2026/codex-now-tackles-long-tasks-with-new-goals-feature>

TechCrunch. “The token bill comes due: Inside the industry scramble to manage AI’s runaway costs.” June 5, 2026. <https://techcrunch.com/2026/06/05/the-token-bill-comes-due-inside-the-industry-scramble-to-manage-ais-runaway-costs/>

TechCrunch. “Are AI tokens the new signing bonus, or just a cost of doing business?” March 21, 2026. <https://techcrunch.com/2026/03/21/are-ai-tokens-the-new-signing-bonus-or-just-a-cost-of-doing-business/>

Tech Jacks Solutions. “Claude Fable 5 Review: Anthropic Mythos-Class Flagship (2026).” June 2026. <https://techjacksolutions.com/ai-tools/anthropic-claude/claude-fable-5-review/>

The AI Consulting Network. “AI Sticker Shock: The Enterprise AI ROI Reckoning.” May 2026. <https://www.theaiconsultingnetwork.com/blog/enterprise-ai-roi-sticker-shock-cre-investors-2026>

Vaasblock. “Enterprise AI Spending ROI Crisis 2026.” June 2026. <https://www.vaasblock.com/news/corporate-ai-spending-roi-enterprise-reckoning-2026/>

VentureBeat. “Anthropic says it hit a \$30 billion revenue run rate after ‘crazy’ 80x growth.” May 2026. <https://venturebeat.com/technology/anthropic-says-it-hit-a-30-billion-revenue-run-rate-after-crazy-80x-growth>

Verdent. “Grill-Me + Trellis: The Vibe Coding Workflow That Replaced My Planning Phase.” May 2026. <https://www.verdent.ai/guides/grill-me-trellis-vibe-coding-workflow>

Wiegold, T. “The Ralph Loop: How Recursive AI Agents Actually Work.” May 2026. <https://thomas-wiegold.com/blog/ralph-loop-how-recursive-ai-agents-work/>

Wiggum CLI. “Ralph Loop — Autonomous Coding Technique.” 2026. <https://wiggum.app/ralph-loop/>

Willison, S. “Anthropic’s run-rate revenue hits \$47 billion.” May 29, 2026. <https://simonwillison.net/2026/May/29/anthropic/>

Yegge, S. “Welcome to Gas Town.” January 1, 2026. <https://steve-yegge.medium.com/welcome-to-gas-town-4f25ee16dd04>

Yegge, S. “The Future of Coding Agents.” January 2026. <https://steve-yegge.medium.com/the-future-of-coding-agents-e9451a84207c>

Zitron, E. “Anthropic’s ‘Profitability’ Swindle.” May 2026. <https://www.wheresyoured.at/anthropics-profitability-swindle/>

The Intelligence Economy · Report 3

© 2026 Wisdom Hill. Licensed under CC BY-NC-ND 4.0.

Quotation and summary are freely permitted with attribution.

Cite as: Wisdom Hill Research. 2026. “3. The Token Economics of Coding Agents.”

In The Intelligence Economy, No. 3. Wisdom Hill.

<https://wisdomhill.github.io/intelligence-economy/>